Andromeda Rust 2026 Transverse Architecture Plan

Hardware-aware Rust engineering, four-engine composition, and cross-cutting implementation doctrine

OpenAI GPT-5.5 Pro

May 7, 2026

Andromeda Rust 2026 Transverse Architecture Plan

Document status: Consolidated architecture guidance
Audience: Andromeda maintainers, Rust engine developers, database architects, security reviewers, performance engineers, release owners
Language target: American English
Style target: Microsoft technical documentation style
Repository cross-check target: `AriusII/Andromeda`, branch `main`
Date: May 7, 2026

In this article

This article defines how Andromeda should use Rust in 2026 as an end-to-end engineering discipline. It consolidates internal project doctrine, repository state, Rust 2026 ecosystem facts, hardware research, and database-engine constraints into one practical implementation plan.

You will learn how to:

- Apply Rust 2024 Edition safely to a mission-critical database engine.
- Structure the codebase around four engine clusters and cross-cutting modules.
- Preserve Andromeda’s contract-first, procedure-only, WAL-first doctrine.
- Use CPU, GPU, RAM, NVMe, SSD, and HDD as explicit policy-controlled resources.
- Keep GPU, learned components, predictive evidence, and hardware acceleration outside the C5 commit and recovery path.
- Define concrete acceptance criteria, CI gates, ADRs, test matrices, and roadmap stages.

> [!IMPORTANT] Rust is not Andromeda’s value proposition by itself. The value comes from using Rust as a strict systems language to reduce ambiguity, constrain unsafe code, encode contracts, prove recovery, and make critical decisions observable.

Executive summary

Andromeda should use Rust as its primary implementation language because the project needs memory safety without a global garbage collector, explicit ownership, deterministic resource boundaries, low-level I/O control, strong type modeling, FFI access, CPU intrinsics, and disciplined concurrency.

The correct 2026 posture is not “use Rust because Rust is modern.” The correct posture is:

```
Rust 2024 Edition
+ explicit MSRV policy
+ strict Cargo workspace governance
+ typed engine boundaries
+ explicit binary codecs
+ narrow audited unsafe
+ deterministic WAL/recovery tests
+ policy-controlled hardware acceleration
+ observable adaptive decisions
```

The repository already contains a meaningful foundation: a Rust 2024 workspace, modular crates, a local recoverable vertical prototype, SRPL components, typed Procedure contracts, transaction/WAL/storage foundations, protocol surfaces, observability, benchmark scaffolding, IAM primitives, and HA/DR/PITR contracts. It is not yet a production SGBDRT runtime, a complete QUIC application server, or a complete durable storage engine. The next milestone is not GPU or learned optimization. The next milestone is a durable, catalog-backed, recoverable procedure path:

```
Procedure
-> Catalog
-> SRPL IR
-> Admission
-> Transaction
-> WAL
-> Heap/Page
-> Recovery
-> ResultStream
```

The final architecture should be organized into four engine clusters:

Engine cluster	Purpose	Criticality posture
Contract and Language Engine	Catalog, Type System, Procedure contracts, SRPL, StructuredObjects, Maps definitions.	Strict, versioned, compile-time oriented.

Engine cluster	Purpose	Criticality posture
Transactional Truth Engine	Transaction Kernel, WAL, MVCC, Storage, BufferPool, HotStore, ColdStore, manifests, recovery.	C5 for commit truth and recovery.
Surface and Governance Engine	QUIC/RPC, Protobuf frames, IAM, security, audit, admin, HA/DR, backup, PITR, forensic.	Fail-closed, audited, surface-separated.
Adaptive Intelligence and Hardware Engine	Procedure Store, optimizer, statistics, ScenarioEvidence, analytics, CPU/GPU/NVMe/HDD policies.	Advisory, bounded, explainable, disableable.

The cross-cutting modules are more important than the names of individual crates. The essential planes are Durability, Security, Contract, Policy, Resource Governance, Observability, Compatibility, Testing, Supply Chain, and Hardware Abstraction.

Source cross-check

This plan was cross-checked against four source categories.

Internal Andromeda doctrine

The internal Andromeda documentation defines a stable doctrine:

- No ad hoc SQL application surface.
- Every application execution goes through a cataloged Procedure.
- Every Procedure has a typed, hashed, versioned contract.
- Every Procedure is transactionally scoped.
- No visible commit without durable WAL.
- RAM is never system truth.
- GPU never participates in commit, rollback, WAL, recovery, MVCC visibility, or security-critical paths.
- Predictive evidence never decides alone.
- Active plans are tied to `CatalogVersion + StatsVersion + ContractHash`.
- Every critical decision must be observable and explainable after the fact.

Primary internal sources:

- `00_ANDROMEDA_INDEX_ET_MODE_DE_LECTURE.md`
- `01_DOCTRINE_LEXIQUE_ARCHITECTURE_CATALOGUE_MODELIZATION.md`
- `02_TYPE_SYSTEM_SRPL_PROCEDURES_MAPS.md`
- `03_TRANSACTION_WAL_MVCC_STORAGE_RECOVERY.md`
- `04_QUIC_RPC_SECURITY_HADR_OPERATIONS.md`
- `05_OPTIMIZER_STATS_ANALYTICS_HARDWARE_ROADMAP_SOURCES.md`
- `Moteur de stockage sécurisé.txt`
- `Rapport consolidé sur les SGBDR, ACID, les transactions, l'algebre relationnelle, la normalisation, les statistiques et l'indexation.pdf`

- `Corpus de reference pour les SGBDR, ACID, l'algebre relationnelle, la normalisation, les statistiques et les structures de recherche.pdf`
- `Fondation de SRPL pour un langage procedural relationnel strict.pdf`
- `Recherche processeurs et Rust.txt`
- `Recherche GPU Rust 2026.txt`
- `Andromeda Rust Engineering Doctrine 2026`

Repository state

The GitHub `main` branch currently exposes:

- A Rust 2024 workspace with `resolver = "3"`.
- Current repository MSRV set to `rust-version = "1.85"`.
- Crates for benchmark, CLI, catalog, core, execution, observability, protocol, QUIC, SRPL, storage, and transaction.
- Dependencies including `quinn`, `rustls`, `rcgen`, `tokio`, and `proptest`.
- A README that identifies Andromeda as a modern relational transactional database project with a strict native surface: `QUIC + custom typed RPC + cataloged Procedure + SRPL + typed ResultStream`.
- A current-state document that classifies the repository as a recoverable vertical prototype with a broad modular foundation, not a production runtime.
- A roadmap that prioritizes durable Procedure execution before optimizer, analytics, GPU, and HA/DR production operations.

Rust 2026 ecosystem

As of May 7, 2026, the current stable Rust release checked for this document is Rust 1.95.0, published on April 16, 2026. Rust 2024 Edition is the right edition baseline for Andromeda, but the project should not force a MSRV bump from `1.85` to `1.95` without an Architecture Decision Record (ADR), CI update, dependency audit, and release note.

Database science and hardware research

The database science sources confirm that the durable core remains relational algebra, cost-based optimization, statistics, transactions, WAL/ARIES-style recovery, dependency-guided normalization, B/B+Trees, and explicit isolation semantics. The modern 2024-2026 frontier - learned cardinality estimation, learned indexes, learned optimizers, deterministic concurrency, GPU analytics - is useful, but not a universal replacement for the classical foundation.

---

1. Baseline decisions

1.1 Two-level Rust baseline

Use a two-level Rust baseline.

Baseline	Version	Purpose	Policy
Repository MSRV	Rust 1.85	Current `main` branch compatibility and Rust 2024 Edition minimum.	Keep until an ADR explicitly changes it.
2026 engineering toolchain	Rust 1.95.0	Current stable toolchain for development validation, benchmarking, and future adoption.	Use in CI advisory lane and local validation; promote to MSRV only after ADR.

This avoids an architectural error: using the latest stable release as a hidden compatibility requirement while the repository still declares Rust 1.85.

Recommendation

Create `ADR-0001-rust-baseline-and-msrv.md` with this content:

```
Current production MSRV: 1.85
Current development stable baseline: 1.95.0
Edition: 2024
Workspace resolver: 3
Production bump policy: ADR + CI + dependency audit + release note
Nightly policy: tooling-only unless separately approved
```

1.2 Rust 2024 Edition rules that matter

Rust 2024 is useful for Andromeda because it makes safety boundaries more visible.

Rule	Why it matters for Andromeda
`unsafe extern` blocks are required.	GPU, OS, storage, crypto, and low-level platform APIs must explicitly mark FFI risk.
Unsafe attributes require `unsafe(...)`.	`no_mangle`, `export_name`, and `link_section` cannot hide linker-level soundness risk.
`unsafe_op_in_unsafe_fn` is visible.	Unsafe functions do not silently make every internal unsafe operation acceptable.
`edition = "2024"` implies resolver 3.	Cargo can take `rust-version` compatibility into dependency resolution.
Stable custom target JSON support was removed in Rust 1.95.	Do not depend on stable custom target specs for production binaries.

Engineering rule

Every crate must inherit these root lints:

```toml
[workspace.lints.rust]
unsafe_op_in_unsafe_fn = "deny"
unreachable_pub = "warn"
missing_docs = "warn"

[workspace.lints.clippy]
correctness = "deny"
```

```
suspicious = "deny"
perf = "warn"
complexity = "warn"
style = "warn"
```

1.3 Stable production, nightly tooling

Use stable Rust for production binaries. Allow nightly only for tooling lanes:

```
Miri
cargo-fuzz
sanitizers
experimental CPU/GPU research
format or lint experiments
compiler diagnostics exploration
```

Never make nightly-only Rust features a hidden runtime requirement for C5 paths.

---

2. Architecture doctrine mapped to Rust

2.1 Strict boundaries

Andromeda boundary	Rust implementation rule
Type System	Use domain newtypes, checked constructors, non-zero types where appropriate, explicit codecs, and no implicit conversions.
Procedure Contract	Represent contracts as immutable descriptors. Hash canonical binary or canonical IR forms, not source text.
Catalog	Use append-only change records, monotonic versions, and deterministic dependency graphs.
SRPL	Separate parser, binder, semantic IR, lowering, and execution adapter.
Transaction	Encode state transitions with enums, typestate where useful, and runtime guards for I/O-dependent states.
WAL	Use explicit record codecs, LSN checks, CRC/hash chain, durable flush results, and crash tests.
Storage	Never serialize Rust-native structs directly. Use verified pages, segments, manifests, and recovery reports.
Network	QUIC is transport only. Andromeda RPC defines semantics.
Security	Separate certificate identity, user principal, roles, permissions, policies, and audit records.
Import	Apply `DefinitionBatch` through dry-run, dependency ordering, transactional catalog changes, and WAL coverage.

Andromeda boundary	Rust implementation rule
Recovery	Any persisted effect must be replayable, reconstructible, or explicitly outside system truth.

2.2 Adaptive internals

Adaptive area	Allowed	Required guardrail
Optimizer	Plan choice, join order, access path, PlanClass.	DecisionTrace, cost terms, versions, hysteresis.
Plan Cache	Multi-plan by bounded classes.	Quotas, eviction, no unbounded specialization.
Statistics	Histogram refresh, skew detection, candidate publication.	`StatsVersion`, validation, controlled switch.
Procedure Store	Runtime feedback and regression detection.	Observes; does not decide alone.
ScenarioEvidence	Benchmarks, simulations, prediction evidence.	Non-authoritative, expirable, tied to versions.
CPU acceleration	SIMD, crypto acceleration, CRC/hash acceleration.	Runtime detection, scalar fallback, tests.
GPU acceleration	Batch statistics, analytics, Maps, vector extensions.	Outside commit/recovery; CPU fallback; kill switch.
Backpressure	Batch shrink, spool, reject, slow producer.	Policy-bound, observable, no WAL starvation.

2.3 Feature acceptance gate

A feature can enter the core only when it passes all checks.

Criterion	Required question	Reject or sandbox when
Definability	Can the behavior be specified without ambiguity?	It depends on ambient runtime magic.
Determinism	Same input + same state + same versions -> same result?	It depends on unseeded randomness or physical order.
Typing	Is the shape known before execution?	It returns dynamic records or string-built shapes.
Boundedness	Can CPU/RAM/NVMe/GPU/temp usage be limited?	It can run without resource budget.
Observability	Can the effect be traced and measured?	It changes behavior invisibly.
Recovery	Can it survive crash or be reconstructed?	It mutates durable state outside WAL.
Security	Can IAM and audit constrain it?	It bypasses surface, permission, or policy.

Criterion	Required question	Reject or sandbox when
Versioning	Is it tied to catalog, policy, statistics, or format version?	It changes semantics without a version.
Explainability	Can a post-mortem explain it?	It is a black-box forced decision.
Disablement	Can it be turned off safely?	It changes commit truth or storage truth.

---

3. Four-engine architecture

Andromeda’s detailed documents list many engines and planes. For implementation governance, group them into four engine clusters. This reduces integration drift without collapsing everything into a God Engine.

3.1 Engine 1 - Contract and Language Engine

Purpose

Define what exists, what can be called, what shape it has, what it reads and writes, what permissions it requires, and how SRPL source becomes typed semantic IR.

Included modules

```
Catalog & Contract Engine
Type System
SRPL Compiler
StructuredObject descriptors
Procedure descriptors
Map descriptors
Enum descriptors
DefinitionBatch pipeline
Compatibility policy
Client SDK contract generator
```

Current crates

```
andromeda-catalog
andromeda-srpl
andromeda-proto
andromeda-core
```

Required Rust posture

- Use immutable descriptors for published catalog objects.
- Use newtypes for all identifiers and versions.
- Use canonical serialization for `ContractHash` and `IrHash`.
- Keep `DefinitionBatch` application transactional and WAL-covered.
- Refuse incomplete catalog publication.

- Keep SRPL parser, binder, IR, lowering, and interpreter or execution adapter separated.

Required acceptance checks

```
ContractHash is deterministic.
CatalogVersion is monotonic.
DefinitionBatch rollback is all-or-nothing.
Invalid SRPL fails dry-run before publication.
optional one requires branch handling.
one requires uniqueness proof or runtime cardinality check.
No SELECT *.
No dynamic SQL text.
No hidden nullable semantics.
```

3.2 Engine 2 - Transactional Truth Engine

Purpose

Make mutations real, durable, visible, recoverable, and explainable.

Included modules

```
Transaction Kernel
WAL / LogStream
MVCC
Lock manager
BufferPool
DiskPageStore
HotStore NVMe
ColdStore HDD
Manifest chain
SegmentIndex
Checkpoint manager
Recovery manager
Crash runner
```

Current crates

```
andromeda-tx
andromeda-storage
andromeda-core
andromeda-observe
```

Required Rust posture

- Commit visibility must be represented by a typed durable WAL result.
- Page writes must know their `PageLsn` and observed durable LSN.
- BufferPool must use pin guards and explicit dirty tracking.
- WAL parsing must be fuzzed.
- Page and manifest parsing must be fuzzed.
- Recovery must produce a structured `RecoveryReport`.

Required acceptance checks

```
Crash before WAL flush -> mutation invisible.
Crash after WAL flush before page flush -> mutation visible after REDO.
WAL truncated tail -> stop at last valid record.
WAL middle corruption -> forensic or restore path.
Manifest corruption -> previous manifest or ForensicOnly.
Snapshot staging crash -> no published half-snapshot.
BufferPool dirty page never implies durability.
```

3.3 Engine 3 - Surface and Governance Engine

Purpose

Control how the outside world touches Andromeda, enforce identity and permissions, and prove operational decisions.

Included modules

```
QUIC surfaces
Custom typed RPC
Protobuf frame envelopes
mTLS
CertificateIdentity
UserPrincipal
Roles and permissions
Policies
Audit ledger
Admin Surface
HA/DR Cluster Surface
Backup/restore/PITR
ForensicStart
Runbooks
```

Current crates

```
andromeda-proto
andromeda-quic
andromeda-exec
andromeda-observe
andromeda-storage
andromeda-core
```

Required Rust posture

- Use QUIC as transport only.
- Keep RPC semantics in `andromeda-proto` and application logic in execution/catalog crates.
- Validate contract, surface, and IAM before transaction creation.
- Separate application, administration, and cluster surfaces.
- Make audit append-only and checksum chained.
- Refuse raw page/WAL inspection on Application Surface.

Required acceptance checks

```
Wrong surface -> no transaction.
Disabled principal -> no transaction.
Missing permission -> no transaction.
ContractHash mismatch -> no transaction.
Admin frame on Application Surface -> ProtocolError or PermissionError.
Audit cannot be globally disabled.
Result metadata precedes payload.
Backpressure cannot starve WAL flush.
```

3.4 Engine 4 - Adaptive Intelligence and Hardware Engine

Purpose

Make Andromeda fast without making it unpredictable.

Included modules

```
Procedure Store
Statistics Engine
Optimizer
Plan Cache
ScenarioEvidence
Benchmark Engine
Analytical Maps
Columnar segments
CPU profile manager
GPU batch runtime
NVMe/SSD scheduler
HDD ColdStore scheduler
Resource governance
```

Current crates

```
andromeda-bench
andromeda-catalog
andromeda-core
andromeda-storage
andromeda-observe
future: andromeda-optimizer
future: andromeda-analytics-gpu
```

Required Rust posture

- Treat learned and predictive components as evidence, not truth.
- Keep plan classes bounded.
- Keep statistics versioned.
- Use CPU scalar fallback for every SIMD path.
- Use CPU fallback for every GPU path.
- Use GPU only for batches outside C5 paths.
- Collect hardware usage metrics.

Required acceptance checks

```
ScenarioEvidence::is_authoritative() remains false.
PlanCacheKey includes ProcedureId + ContractHash + CatalogVersion + StatsVersion + PolicyVersion +
PlanClass + ShapeFingerprint.
StatsVersion candidate cannot become active without validation.
GPU job cancellation cannot affect transaction state.
GPU crate cannot be imported by commit/WAL/recovery/security-critical modules.
```

---

4. Cross-cutting modules

4.1 Durability Plane

The Durability Plane enforces:

```
WAL append
WAL flush
commit visibility
page flush eligibility
checkpoint
manifest switch
snapshot publication
WAL retention
restore/PITR
forensic startup
```

Rust implementation pattern

```rust
pub struct DurableCommitEvidence {
    pub tx_id: TxId,
    pub commit_lsn: Lsn,
    pub durable_lsn: Lsn,
    pub flush_latency_micros: u64,
    pub durability_mode: DurabilityMode,
}
```

A transaction must not transition to visible committed state without a `DurableCommitEvidence` value.

4.2 Security Plane

The Security Plane enforces:

```
mTLS -> CertificateIdentity -> UserPrincipal -> Roles -> Permissions -> Policies -> Audit
```

Rust implementation pattern

```rust
pub struct AdmissionDecision {
    pub invocation_id: InvocationId,
```

```
    pub principal_id: PrincipalId,
    pub certificate_id: CertificateId,
    pub surface: SurfaceKind,
    pub procedure_id: ProcedureId,
    pub contract_hash: ContractHash,
    pub policy_version: PolicyVersion,
    pub outcome: AdmissionOutcome,
}
```

`AdmissionOutcome::Denied` must emit audit evidence and must not create a transaction.

4.3 Contract Plane

The Contract Plane enforces:

```
ProcedureName
ContractHash
InputShape
StructuredObjectContracts
OutputShape
ResultStreamMetadata
RequiredPermissions
ReadSet
WriteSet
IsolationPolicy
ResourcePolicy
ProtocolLayout
CompatibilityPolicy
CatalogVersion
PolicyVersion
StatsVersion
```

Rust implementation pattern

```
pub struct ProcedureContractBinding {
    pub procedure_id: ProcedureId,
    pub contract_hash: ContractHash,
    pub catalog_version: CatalogVersion,
    pub stats_version: StatsVersion,
    pub policy_version: PolicyVersion,
    pub manifest_hash: ManifestHash,
}
```

This binding should become mandatory for execution, audit, protocol manifests, benchmark evidence, and plan cache.

4.4 Resource Governance Plane

The Resource Governance Plane enforces budgets:

```
CPU time
RAM bytes
BufferPool pins
TempBytes
SpillBytes
WAL bytes
NVMe queue share
```

```
GPU job count
GPU memory
ResultStream rows and bytes
RPC duration
```

Rust implementation pattern

```rust
pub struct ResourceBudget {
    pub max_duration_millis: u64,
    pub max_temp_bytes: u64,
    pub max_spill_bytes: u64,
    pub max_result_rows: Option<u64>,
    pub gpu_policy: GpuExecutionPolicy,
}
```

Every long-running operation should receive a budget object rather than reading global limits implicitly.

4.5 Observability Plane

The Observability Plane must produce:

```
ProcedureInvocationTrace
TransactionTrace
PlanDecisionTrace
CatalogChangeTrace
DefinitionBatchApplyTrace
SecurityAuditTrace
AdminOperationTrace
RecoveryTrace
ClusterEventTrace
BenchmarkEvidenceTrace
GpuExecutionTrace
```

Rust implementation pattern

```rust
#[tracing::instrument(
    name = "transaction.commit",
    skip_all,
    fields(tx_id = ?tx_id, contract_hash = ?contract_hash)
)]
pub fn commit(&mut self, evidence: DurableCommitEvidence) -> Result<(), TransactionError> {
    // commit transition
    Ok(())
}
```

4.6 Policy Plane

Policies must be versioned. Do not allow invisible behavioral defaults.

```
SecurityPolicy
StoragePolicy
TransactionPolicy
ResourcePolicy
CompatibilityPolicy
```

```
MapConsistencyPolicy
AuditPolicy
RecoveryPolicy
HardwarePolicy
StatisticsPolicy
OptimizerPolicy
```

A critical operation without `PolicyVersion` is incomplete.

4.7 Testing and Evidence Plane

Every engine must define:

```
unit tests
property tests
fuzz targets
crash tests
integration tests
benchmarks
release gates
forensic validation
```

Use tests as evidence, not as ceremony.

---

5. Repository architecture

5.1 Current workspace

The current workspace uses these crates:

```
andromeda-bench
andromeda-cli
andromeda-catalog
andromeda-core
andromeda-exec
andromeda-observe
andromeda-proto
andromeda-quic
andromeda-srpl
andromeda-storage
andromeda-tx
```

This is a good V0 structure. Do not reorganize it aggressively before the durable vertical path is complete.

5.2 Recommended future splits

Split only when responsibility pressure becomes real.

Future crate	Split from	Reason
`andromeda-optimizer`	`andromeda-catalog`	Physical planning and costing will become large enough to own a crate.
`andromeda-statistics`	`andromeda-catalog`	Statistics publication, histograms, NDV, and skew can be isolated.
`andromeda-contract`	`andromeda-catalog` and `andromeda-proto`	Contract types should be reusable by catalog, protocol, execution, and client SDK generation.
`andromeda-codec`	`andromeda-core` and storage/proto modules	WAL/page/manifest/protocol codecs should share checked binary primitives.
`andromeda-analytics`	storage/catalog/bench	Columnar Maps and analytical operators should not leak into OLTP execution.
`andromeda-gpu`	analytics/statistics	Optional GPU runtime must be isolated from C5 modules.
`andromeda-security`	core/exec/quic/observe	IAM and security policies will become large enough to need a dedicated boundary.
`andromeda-admin`	cli/storage/catalog/observe	Backup, restore, debug, maintenance, and DefinitionBatch operations need admin semantics.

5.3 Composition root

The composition root should be thin.

```
andromeda-engine or andromeda-runtime
```

Allowed:

```
configuration
component wiring
startup sequencing
shutdown sequencing
surface registration
health orchestration
feature-gated runtime construction
```

Forbidden:

```
business rules
page parsing
WAL record parsing
contract validation logic
plan search logic
permission bypass
hidden global mutable state
```

---

6. Rust implementation rules

6.1 Newtypes for critical identifiers

Use semantic newtypes instead of raw primitives.

```rust
#[derive(Clone, Copy, Debug, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub struct Lsn(u64);

impl Lsn {
    pub const ZERO: Self = Self(0);

    pub fn new(value: u64) -> Self {
        Self(value)
    }

    pub fn get(self) -> u64 {
        self.0
    }

    pub fn next(self) -> Option<Self> {
        self.0.checked_add(1).map(Self)
    }
}
```

Use newtypes for:

```
DatabaseId
NamespaceId
ObjectId
ProcedureId
InvocationId
TxId
Lsn
CatalogVersion
StatsVersion
PolicyVersion
ContractHash
PlanId
SnapshotId
SegmentId
PageId
TraceId
PrincipalId
CertificateId
SurfaceId
```

6.2 Typestate for limited state transitions

Typestate is useful for small, stable state transitions.

```rust
pub struct Transaction<S> {
    tx_id: TxId,
    state: S,
}

pub struct Created;
```

```rust
pub struct Active;
pub struct Committing;
pub struct Committed;

impl Transaction<Created> {
    pub fn activate(self) -> Transaction<Active> {
        Transaction { tx_id: self.tx_id, state: Active }
    }
}
```

Do not overuse typestate for I/O-dependent transitions. Recovery, commit flush, cancellation, and resource failure require runtime state machines.

6.3 Explicit absence

Use `Option<T>` for local Rust absence. Use domain enums when absence has protocol, business, or diagnostic meaning.

```rust
pub enum CustomerLookup {
    Found(CustomerRecord),
    NotFound,
}
```

Do not map SRPL `optional one Customer` into unchecked `Customer`. The compiler and contract layer must force branch handling.

6.4 Decimal and float discipline

Type family	Allowed	Forbidden
Decimal	Money, accounting, exact quantities, tax, exact contractual rates.	Missing precision/scale, hidden rounding.
Integer	IDs, LSNs, page IDs, counters, row counts, sizes.	Unchecked persisted arithmetic.
Float	Statistics, scores, measurement, analytics, vector workloads.	Keys, foreign keys, WAL consensus values, exact invariants, money.

Recommendation

Use a controlled decimal type with:

```
precision
scale
rounding policy
overflow policy
serialization policy
comparison policy
```

Use explicit NaN policy for floats stored in catalog, statistics, or analytical Maps.

6.5 Error model

Do not use string-only errors.

Error family	Example	Boundary behavior
BusinessError	`InsufficientStock`	Typed Procedure result, rollback if mutation started.
ContractError	`ContractHashMismatch`	Reject before transaction.
PermissionError	`ExecuteProcedureDenied`	Reject before transaction and audit.
CardinalityError	`ExpectedOneFoundMany`	Deterministic fail.
ConstraintError	`UniqueViolation`	Rollback.
ResourceError	`TempBytesQuotaExceeded`	Fail controlled, rollback.
TransactionError	`SerializationFailure`	Retry-aware typed error.
StorageError	`WalRecordCrcMismatch`	Recovery or forensic path.
SystemError	`IoFailure`	Poison/rollback/recovery path.

Use typed errors in library crates. Use `anyhow` only in CLI, tooling, or orchestration where semantic error families have already been preserved.

6.6 Panic policy

Allowed panic contexts:

```
unit tests
property tests
fuzz harness assertions
startup validation before accepting traffic
compile-time impossible branches with proof
```

Forbidden panic contexts:

```
WAL append
commit visibility
recovery replay
RPC request handling
security authorization
storage page parser on untrusted bytes
catalog publication
```

Production `panic = "abort"` is acceptable only after critical errors are typed and surfaced correctly.

---

7. Unsafe Rust policy

7.1 Default rule

`unsafe` is allowed only when it materially improves correctness, performance, or interoperability and cannot be reasonably expressed in safe Rust.

Expected `unsafe` zones:

```
binary page layout wrappers
mmap or direct I/O wrappers
FFI to OS, crypto, QUIC, GPU, or hardware libraries
SIMD intrinsics
custom allocators
lock-free structures
atomic memory ordering internals
```

7.2 Unsafe module structure

Use private unsafe modules with safe public APIs.

```
src/page/
  mod.rs          # safe API
  raw.rs          # private unsafe primitives
  codec.rs        # checked serialization
  validation.rs   # invariant checks
  tests.rs        # unit tests
```

7.3 Safety comment rule

Every unsafe block must include a local `SAFETY:` explanation.

```rust
pub unsafe fn page_from_aligned_ptr<'a>(ptr: *const u8) -> &'a [u8; PAGE_SIZE] {
    // SAFETY:
    // - The caller guarantees that ptr is non-null.
    // - The caller guarantees that ptr is aligned for [u8; PAGE_SIZE].
    // - The caller guarantees that ptr..ptr+PAGE_SIZE is a live allocation.
    // - The returned reference does not outlive the mapped page pin.
    unsafe { &*(ptr.cast::<[u8; PAGE_SIZE]>()) }
}
```

7.4 Forbidden unsafe patterns

```
transmute Rust structs to disk bytes
reinterpret untrusted bytes as typed structs without validation
repr(Rust) on persisted or network formats
usize/isize in persisted or network formats
raw enum discriminants on disk without versioned codec
unchecked pointer arithmetic without bounds proof
creating references to packed fields
using target-feature code without dispatch guard
```

```
using static mut for shared runtime state
calling std::env::set_var after multi-threaded startup
```

7.5 Unsafe review checklist

Check	Required evidence
Boundary	Unsafe API is private or sealed behind safe constructors.
Preconditions	`# Safety` section for unsafe functions.
Invariants	Module-level invariants documented.
Miri	Applicable tests pass under Miri.
Fuzz	Byte parsers and decoders have fuzz targets.
Sanitizers	Nightly sanitizer job covers feasible paths.
Property tests	Roundtrip and invalid-input properties exist.
Audit	Unsafe-specific review approval exists.
Telemetry	Runtime failure mode is observable where relevant.

---

8. Binary format and serialization

8.1 Canonical format

Use explicit little-endian encoders and decoders.

```
u8/u16/u32/u64/u128: little-endian
signed integers: little-endian two's complement
bool: u8 with 0 or 1 only
string: length-prefixed UTF-8 with validation
bytes: length-prefixed or fixed-length by schema
optional: explicit tag
```

Do not serialize Rust-native structs directly.

8.2 Persistent format rules

Rule	Requirement
Magic	Every file, segment, page, and WAL segment has magic bytes.
Version	Major/minor versions and feature flags.
Bounds	Every length and offset checked before allocation/read.
CRC/hash	Every critical persisted block has integrity metadata.
LSN	WAL-covered pages carry PageLSN or equivalent.
Canonicalization	Hash canonical forms, not debug formatting.

Rule	Requirement
Alignment	On-disk parsing must not require Rust-native alignment.

8.3 Codec trait pattern

```rust
pub trait Encode {
    fn encoded_len(&self) -> usize;
    fn encode(&self, out: &mut Vec<u8>) -> Result<(), CodecError>;
}

pub trait Decode<'a>: Sized {
    fn decode(input: &'a [u8]) -> Result<Self, CodecError>;
}
```

For hot paths, use a checked cursor and avoid allocation while parsing fixed headers.

```rust
pub struct DecodeCursor<'a> {
    input: &'a [u8],
    offset: usize,
}
```

8.4 ContractHash rule

`ContractHash` must be computed from a canonical representation containing at least:

```
ProcedureId or qualified name
input shape
StructuredObject descriptors
output shape
ResultStream metadata
required permissions
read/write sets
isolation policy
resource policy
protocol layout
compatibility policy
CatalogVersion scope
PolicyVersion scope
semantic version of hash algorithm
```

Do not hash raw SRPL text. Formatting must not change contract identity.

---

9. Storage architecture

9.1 Recommended physical format

Use a segmented, binary, append-only, immutable cold format with log-structured hot storage:

```
Root pointer A/B
+ signed manifest chain
```

```
+ WAL segments append-only
+ HotStore segments on NVMe
+ ColdStore snapshot segments on HDD
+ compact SegmentIndex
+ audit and archive files
```

Avoid both extremes:

Rejected design	Reason
One monolithic file per database	Hard startup, maintenance, scrub, partial corruption, compaction, and restore.
One file per table/page/index	File count explosion, noisy filesystem metadata, slow startup, difficult backup/snapshot.

9.2 Truth rule

Andromeda must never trust a file alone.

```
System truth = valid root pointer
             + signed manifest
             + verified SegmentIndex
             + published Cold Snapshot
             + durable WAL since RequiredWalStartLsn
```

RAM is not truth. BufferPool is not truth. GPU is not truth. HotStore is reconstructible. ColdStore plus durable WAL is truth.

9.3 Segment families

Family	Extension	Role	Truth status
Root pointer	`.androot`	Redundant pointer to current manifest.	Pointer only.
Manifest	`.andmf`	Signed list of segments, versions, snapshot, WAL range.	Yes.
WAL segment	`.andwal`	Durable append-only transition log.	Yes for recent transitions.
Hot segment	`.andhot`	Hot pages, MVCC versions, delta indexes.	Reconstructible with WAL.
Cold segment	`.andcold`	Immutable published snapshot.	Yes if manifest-published.
Segment index	`.andidx`	Compact page/range/chunk locator.	Verified metadata.
Audit/archive	`.andaudit`, `.andarc`	Audit, WAL archives, backup metadata.	Yes according to policy.

9.4 V0 size defaults

Item	V0 recommendation
Page size	16 KiB
Extent	64 pages = 1 MiB
WAL segment	128 MiB
Hot segment	256 MiB
Cold segment	512 MiB
LOB segment	1 GiB
SegmentIndex target	< 256 MiB

Use 32 KiB pages later for cold analytics or specific storage policies. Do not mix page sizes without explicit format policy.

9.5 WAL record pattern

```
WalRecord {
    Magic,
    RecordLength,
    RecordLengthInv,
    Lsn,
    PrevLsn,
    TxId,
    InvocationId,
    CatalogVersion,
    RecordType,
    ObjectId optional,
    PageId optional,
    PayloadLength,
    Payload,
    Crc64,
    ChainHash
}
```

`RecordLengthInv` is a useful torn-write and misread detector.

9.6 Commit record pattern

```
TxCommit {
    TxId,
    CommitLsn,
    CommitTimestampLogical,
    InvocationId,
    ProcedureId,
    ContractHash,
    CatalogVersion,
    PolicyVersion,
    WriteSetHash,
    LastMutationLsn,
    TxChainHash
}
```

This links RPC, Procedure, contract, transaction, WAL, audit, and recovery.

9.7 HotStore rule

Prefer copy-on-write or log-structured HotStore in V0:

```
mutation page
-> WAL durable
-> new page version in hot segment
-> HotPageMap points to latest version
-> checkpoint
-> merge into cold snapshot
```

This increases write amplification but improves proof quality and recovery determinism.

9.8 ColdStore rule

ColdStore is immutable after publication.

```
fence snapshot
-> build cold segments in staging
-> compute block CRC/hash
-> compute SegmentDirectory
-> compute MerkleRoot
-> seal SegmentTrailer
-> fsync segments
-> build SegmentIndex
-> fsync SegmentIndex
-> write manifest.next
-> fsync manifest
-> switch root slot
-> deferred cleanup
```

9.9 SegmentIndex rule

Startup should be:

```
read root
-> read manifest
-> read SegmentIndex
-> load CatalogRoot
-> load B+Tree roots
-> load HotPageMap checkpoint
-> replay WAL tail
-> rollback incomplete transactions
-> validate invariants
-> open Online, ReadOnly, or ForensicOnly
```

Startup must not scan every cold segment.

---

10. Transaction and recovery

10.1 Transaction states

Use the state set already defined in the project:

```
Created
Active
Committing
Committed
Failed
Poisoned
RollingBack
RolledBack
Disposed
```

`Poisoned` means continuation is unsafe. It must force controlled rollback or recovery handling.

10.2 Commit invariant

```
Visible commit = durable WAL
```

This must hold for:

```
row insert/update/delete
index insert/delete
MVCC version create/close
Map immediate delta
catalog change
security registry mutation
manifest switch
```

10.3 Recovery algorithm

```
1. Read active manifest.
2. Verify signature/hash/CRC.
3. Mount last valid cold snapshot.
4. Read WAL from RequiredWalStartLsn.
5. REDO complete valid records.
6. Identify incomplete transactions.
7. UNDO incomplete transactions.
8. Rebuild hot indexes and Maps when required.
9. Validate catalog, storage, transaction, and security invariants.
10. Open Online, ReadOnly, or ForensicOnly.
```

10.4 Crash-test matrix

Injection point	Expected result
Before durable `TxBegin`	Transaction does not exist.

Injection point	Expected result
After `TxBegin`, before mutation WAL	Incomplete transaction ignored or rolled back.
After mutation WAL, before `TxCommit`	UNDO or rollback.
After durable `TxCommit`, before client ACK	Transaction is committed after recovery.
After client ACK, before dirty page flush	Transaction visible after WAL REDO.
During manifest switch	Old or new manifest valid; never half-published.
During Map delta apply	Map consistency follows WAL and refresh policy.

10.5 RecoveryReport

```rust
pub struct RecoveryReport {
    pub database_id: DatabaseId,
    pub start_mode: StartMode,
    pub snapshot_id: SnapshotId,
    pub manifest_version: ManifestVersion,
    pub required_wal_start_lsn: Lsn,
    pub last_valid_wal_lsn: Lsn,
    pub redo_records_applied: u64,
    pub transactions_committed: u64,
    pub transactions_rolled_back: u64,
    pub corrupt_records_skipped: u64,
    pub indexes_rebuilt: u64,
    pub maps_validated: u64,
    pub catalog_version_recovered: CatalogVersion,
    pub security_audit_status: SecurityAuditStatus,
    pub open_mode: DatabaseOpenMode,
    pub warnings: Vec<RecoveryWarning>,
    pub errors: Vec<RecoveryError>,
}
```

---

11. SRPL and type system

11.1 SRPL purpose

SRPL is a Strict Relational Procedure Language. It must not become a renamed SQL dialect. It exists to make intent, types, cardinality, absence, transaction policy, read/write sets, and result shapes explicit.

11.2 Required SRPL principles

```
set semantics by default
explicit absence, never ambient NULL
explicit cardinality
strict lexical scope
bounded loops only
transaction as contract
syntax lowered into stable semantic IR
no dynamic SQL text
```

```
no network from Procedure
no external filesystem from Procedure
```

11.3 Cardinality types

Form	Meaning
`one Customer`	Exactly one. Error if zero or many.
`optional one Customer`	Zero or one. Branch handling required.
`many Customer`	Zero to N.
`nonempty many Customer`	At least one.
`collection of T`	Contractual stream or collection.

11.4 Absence rule

```
let maybeCustomer be optional one Customer
    from Customers
    where Customer.Id = CustomerId;

when maybeCustomer has value as Customer do
    return Customer;
otherwise
    fail with CustomerNotFound;
end when;
```

The absence branch is not optional.

11.5 Aggregates rule

Aggregates must define empty-set behavior.

```
compute
    TotalAmount as sum of Order.Amount else 0,
    AverageScore as average of Score else absent,
    MaxAmount as max of Amount else absent,
    CountRows as count of Order;
```

11.6 Mutation rule

```
Insert: constraints + WAL
Update: where required for multi-row mutation
Delete: where required for multi-row mutation
Merge: cardinality contract required
All: RowsAffected available
All: WAL coverage required
```

11.7 Compiler stages

```
SRPL source
-> Lexer/Parser
-> AST
-> Binder
-> Type/cardinality/effect validation
-> Semantic IR / ALT
-> Relational normalization
-> Physical plan candidates
-> Costing and evidence integration
-> Procedure Plan Cache
-> Procedure Code Cache optional
```

No code generation is valid without:

```
ContractHash
CatalogVersion
StatsVersion
PlanId
PolicyVersion
DecisionTraceId
```

---

12. QUIC/RPC and security

12.1 QUIC is transport

QUIC provides secure transport, multiplexed streams, connection state, flow control, and congestion behavior. It does not define Andromeda semantics.

Andromeda semantics live in:

```
custom typed RPC
Protobuf or canonical binary frames
Procedure contracts
Catalog & Contract Engine
Security/IAM Plane
Execution Engine
```

12.2 Surface separation

Surface	Allowed	Forbidden
Application	HELLO, AUTH, GET_CONTRACTS, EXECUTE_PROCEDURE, STREAM_RESULT, ERROR, minimal telemetry.	Create objects, admin, cluster, debug, backup, restore, raw pages/ WAL.
Administration	DefinitionBatch import, plans, Procedure Store, backup/restore, debug snapshot, certificates, policies.	Unaudited access, security bypass.

Surface	Allowed	Forbidden
HA/DR Cluster	WAL shipping, health, quorum, fencing, manifests, promotion.	Application business operations, auto-promotion without quorum.

12.3 Admission order

```
1. Validate transport/session.
2. Validate surface.
3. Authenticate certificate.
4. Resolve CertificateIdentity.
5. Resolve UserPrincipal.
6. Check principal status.
7. Load roles, groups, permissions.
8. Load policies.
9. Validate ProcedureName + ContractHash.
10. Validate payload shape.
11. Validate resource budget.
12. Emit admission audit evidence.
13. Create TransactionScope only if admitted.
```

12.4 Frame error taxonomy

Family	Examples
TransportError	Stream reset, transport timeout.
ProtocolError	Invalid frame type, invalid header CRC.
AuthError	Expired or revoked certificate.
PermissionError	`ExecuteProcedure` denied.
ContractError	ContractHash mismatch.
PayloadError	Malformed StructuredObject.
ExecutionError	Business error.
TransactionError	Deadlock or serialization failure.
ResourceError	Backpressure, quota, timeout.
SystemError	Storage, recovery, corruption.

12.5 Rust crate boundary

```
andromeda-quic = transport, connection, stream, TLS, flow control
andromeda-proto = frames, envelopes, payload descriptors, errors
andromeda-catalog = contracts and catalog identity
andromeda-exec = admission and orchestration
andromeda-observe = trace and audit events
```

Do not let QUIC stream state decide Procedure semantics.

---

13. Statistics, optimizer, and evidence

13.1 Statistics as an engine component

Statistics are not auxiliary. They drive performance and plan stability.

Minimum `StatsObject`:

```
StatsId
ObjectId
StatsVersion
ColumnSet
RowCount
DistinctCount
AbsentCount
Histogram
Density
SkewScore
CorrelationHints
SampleRate
BuiltAt
ValidatedAt
ValidationState
SourceSnapshotId
```

13.2 Publication lifecycle

```
Candidate
-> Validating
-> Published
-> Expired
-> Superseded
```

Rejected candidate statistics must not affect active plans.

13.3 Cost model V0

```
TotalCost = CpuCost
          + LogicalIoCost
          + PhysicalIoCost
          + WalCost
          + TempCost
          + NetworkCost
          + RiskPenalty
```

Term	Inputs
CpuCost	operators, rows, SIMD availability.
LogicalIoCost	pages, expected buffer hits.
PhysicalIoCost	HotStore/ColdStore, random/sequential, storage temperature.

Term	Inputs
WalCost	WAL bytes, flush policy, mirror or sync replica.
TempCost	sort/hash spill, NVMe pressure.
NetworkCost	ResultStream bytes, batch size, client speed.
RiskPenalty	stale stats, cardinality uncertainty, skew, plan instability.

13.4 Plan cache key

```rust
pub struct PlanCacheKey {
    pub procedure_id: ProcedureId,
    pub contract_hash: ContractHash,
    pub catalog_version: CatalogVersion,
    pub stats_version: StatsVersion,
    pub policy_version: PolicyVersion,
    pub plan_class: PlanClass,
    pub shape_fingerprint: ShapeFingerprint,
}
```

13.5 Plan classes

```
Generic
Small
Medium
Large
Skewed
StructuredObjectSmall
StructuredObjectLarge
Maintenance
```

Plan classes are bounded. Do not create one plan per arbitrary input value.

13.6 ScenarioEvidence

ScenarioEvidence can inform optimizer decisions but cannot force them.

```
ScenarioId
ScenarioHash
ProcedureId
ContractHash
CatalogVersion
StatsVersion
PlanClass
EstimatedCardinality
EstimatedLogicalCost
EstimatedIoTierImpact
EstimatedSpillRisk
ConfidenceScore
CriticalityScore
FreshnessScore
RealismScore
StabilityScore
```

```
ValidationState
Expiration
```

Keep scores separate. Do not collapse them into a pseudo-truth scalar.

13.7 Learned components

Component	Status
Learned Cardinality Estimation	C0/C1 experimental, C2 only after validation. Never alone in C5.
Learned Index	Optional analytical experiment. B+Tree fallback required.
Learned Optimizer	Evidence or suggestion only. Final decision governed by optimizer policy.
Workload prediction	Useful for capacity planning and scenario generation. Not transaction truth.
Poisoning defense	Required before historical query data affects planning.

---

14. Hardware-aware Rust

14.1 Hardware stack

Andromeda’s target stack is:

```
CPU x64 and ARM64
GPU for analytics/statistics/vector workloads
RAM
NVMe/SSD hot cache and WAL
HDD cold truth via published snapshots
```

Each tier must have a role and an explicit policy.

14.2 CPU profiles

Architecture	Profiles
x64	`x64-baseline`, `x64-avx2`, `x64-avx512`, `x64-server-amx`
ARM64	`arm64-baseline-neon`, `arm64-sve`, `arm64-sve2`, `arm64-server-secure`

Use runtime feature detection and a hardware profile recorded at startup.

14.3 CPU acceleration rules

Workload	Acceleration	Rule
WAL checksum	CRC/SHA acceleration.	Scalar fallback and test vectors.
Encryption	AES-NI/AES ARM or vetted provider.	No custom cryptography.
Short scans	scalar/SIMD.	Benchmark before accepting.
Bitmap/cardinality	POPCNT/AVX/SVE.	Same results across kernels.
Compression	SIMD if measured.	Budget CPU and no C5 surprise.
Latches/counters	atomics.	Avoid global locks and verify memory ordering.

14.4 Runtime dispatch

```rust
pub trait CrcKernel {
    fn crc64(&self, input: &[u8]) -> u64;
}

pub fn select_crc_kernel() -> &'static dyn CrcKernel {
    #[cfg(target_arch = "x86_64")]
    {
        if std::is_x86_feature_detected!("sse4.2") {
            return &CRC_SSE42;
        }
    }
    &CRC_SCALAR
}
```

Never set `target-cpu=native` globally for distributed binaries.

14.5 SIMD acceptance rule

```
scalar implementation exists
accelerated implementation exists
runtime dispatch guards target feature
same vectors pass on scalar and accelerated path
benchmarks prove benefit
unsafe preconditions are documented
fallback path is exercised in CI
```

14.6 GPU allowed and forbidden work

Allowed:

```
histograms
cardinality estimation batches
skew detection
analytical scans
large aggregations over snapshot-only or deferred Maps
```

```
benchmark scenarios
bounded vector similarity extension
```

Forbidden:

```
commit
WAL append or flush
rollback
recovery
single-row OLTP lookup
short MVCC visibility checks
security-critical authorization
catalog publication
```

14.7 GPU module policy

A GPU runtime must be optional and isolated.

```
crates/andromeda-gpu/          # optional, non-C5
crates/andromeda-analytics/    # CPU-first analytics
crates/andromeda-statistics/   # CPU-first statistics
```

Every GPU job must record:

```
DeviceId
DriverVersion
KernelId
InputShape
CatalogVersion
StatsVersion
PolicyVersion
Duration
TransferBytes
ValidationState
FallbackReason
```

14.8 NVMe/SSD policy

Allowed:

```
WAL durable path
Hot pages
Version store
Temp store
Stats build
Benchmark temp
Procedure code/cache if policy allows
```

Required metrics:

```
wal bytes
hot page bytes
write amplification
wear percentage
spare available
```

```
media errors
unsafe shutdowns
temperature
latency p50/p95/p99
queue depth
```

14.9 HDD policy

Allowed:

```
cold snapshots
archives
cold indexes
published stats
manifests
forensic copies
backup staging
```

Forbidden:

```
transaction direct random update
WAL active unique copy
active version store
temp intensive workloads
commit path
```

---

15. Testing strategy

15.1 Test layers

Layer	Volume	Purpose
Unit tests	Very high	Newtypes, codecs, state transitions, small algorithms.
Property tests	High	Roundtrip, monotonicity, idempotence, invariants.
Fuzz tests	High for parsers/codecs	SRPL, RPC, WAL, pages, manifests, SegmentIndex.
Miri tests	Targeted	Unsafe wrappers, pointer-sensitive logic.
Loom tests	Targeted	Atomics, lock-free internals, pin guards.
Integration tests	Medium	Procedure call through local RPC harness.
Crash/recovery tests	Mandatory	Kill at WAL, page, manifest, catalog boundaries.
HA/DR tests	Critical but scoped	Quorum, fencing, WAL shipping, promotion.

Layer	Volume	Purpose
End-to-end tests	Low but stable	Full cataloged procedure scenario.
Benchmarks	Continuous but separate	WAL throughput, page scan, optimizer latency.

15.2 Required fuzz targets

```
fuzz_targets/srpl_parser.rs
fuzz_targets/rpc_frame.rs
fuzz_targets/structured_object.rs
fuzz_targets/wal_record.rs
fuzz_targets/page_header.rs
fuzz_targets/manifest.rs
fuzz_targets/segment_index.rs
fuzz_targets/contract_hash.rs
```

Fuzz target rule:

```
no panic
no OOM
no unbounded allocation
invalid input returns typed error
no UB under supported sanitizer/Miri lanes
```

15.3 Property tests

```rust
proptest::proptest! {
    #[test]
    fn wal_record_roundtrips(record in any_valid_wal_record()) {
        let encoded = record.encode_to_vec().unwrap();
        let decoded = WalRecord::decode(&encoded).unwrap();
        prop_assert_eq!(record, decoded);
    }
}
```

Required properties:

```
encode/decode roundtrip
invalid length never panics
LSN ordering is monotonic
contract hash is deterministic
catalog dependency sort is stable
page free-space accounting never underflows
StatsVersion publication rejects invalid transitions
ResourceBudget cannot exceed policy maxima
```

15.4 CI gates

Pull request gate

```
cargo fmt --all -- --check
cargo check --workspace --locked
cargo test --workspace --locked
cargo clippy --workspace --all-targets --locked -- -D warnings
```

Extended gate

```
cargo nextest run --workspace --all-features
cargo test --doc --workspace
cargo deny check
cargo audit
cargo vet
```

Nightly safety gate

```
cargo +nightly miri test -p andromeda-core
cargo +nightly miri test -p andromeda-storage
cargo +nightly miri test -p andromeda-tx
cargo +nightly fuzz run wal_record -- -max_total_time=300
cargo +nightly fuzz run rpc_frame -- -max_total_time=300
cargo +nightly fuzz run srpl_parser -- -max_total_time=300
```

Storage/recovery gate

```
crash runner matrix
manifest switch interruption tests
WAL truncation tests
page torn write tests
snapshot restore tests
PITR restore tests
catalog DefinitionBatch crash tests
```

Performance gate

```
record Rust compiler version
record CPU/GPU/NVMe profile
record OS and filesystem
run Criterion or engine-specific benchmarks
compare against baseline
fail only on agreed critical thresholds
store evidence with benchmark run ID
```

---

16. Supply chain and dependency policy

16.1 Baseline tools

Use these tools from the first release gate:

```
cargo audit
cargo deny check
cargo vet
cargo tree -d
cargo metadata
```

16.2 Dependency admission rule

A new dependency must document:

```
why it is needed
which crate owns it
whether it enters C5 path
license
MSRV impact
unsafe usage
transitive dependency impact
security advisory status
alternatives considered
removal plan or disablement plan
```

16.3 C5 dependency rule

Dependencies used in WAL, recovery, storage codec, transaction, IAM, or audit paths require stricter review:

```
minimal dependency graph
known unsafe audit
license accepted
cargo-audit clean or exception documented
cargo-deny clean
cargo-vet audit or exception
no hidden network/filesystem side effects
```

16.4 No implicit runtime JSON

The roadmap says no runtime JSON default for typed protocol payloads. JSON can exist in tooling, diagnostics, test fixtures, or admin exports, but not as the default runtime contract for Procedure payloads.

16.5 No gRPC

The protocol is custom typed RPC over QUIC. Protobuf can be the wire contract format. Do not introduce gRPC or `tonic` as an application surface.

---

17. Implementation roadmap

17.1 P0 - Stabilize current `main`

Deliverables:

```
workspace gates green
current-state document accurate
roadmap accurate
no doctrine regressions
PR-sized integration batches
```

Acceptance:

```
cargo fmt passes
cargo check passes
cargo test passes
cargo clippy passes
no SQL/gRPC/runtime JSON drift
GPU boundary tests pass
```

17.2 P1 - Durable ProductStock vertical path

Deliverables:

```
Inventory.ProductStock durable heap table
Procedure mutates heap/page state
WAL covers mutation
commit visible only after durable WAL
recovery reconstructs stock state
ResultStream metadata precedes payload
Procedure Store records terminal outcome
```

Acceptance:

```
crash before WAL flush -> stock unchanged
crash after WAL flush -> stock recovered
contract mismatch -> no transaction
permission denied -> no transaction
runtime record emitted on commit/fail/abort
```

17.3 P2 - Catalog and DefinitionBatch durability

Deliverables:

```
CatalogStore durable V0
DefinitionBatch dry-run complete
ApplyDefinitionBatch WAL-covered
CatalogVersion publication atomic
catalog recovery replay
contract compatibility policy
```

Acceptance:

```
invalid SRPL rejected in dry-run
breaking change rejected unless policy permits
catalog crash mid-apply publishes no half-catalog
recovery reconstructs last valid catalog
```

17.4 P3 - Generic Procedure execution

Deliverables:

```
ProcedureRegistry catalog-backed
dispatch by ProcedureId + ContractHash
typed payload validation
SRPL IR to minimal physical operators
business/contract/permission/transaction/resource/system errors typed
```

Acceptance:

```
two independent cataloged Procedures execute without hard-coded handler
Admission order enforced before transaction
ResultStream metadata before payload
Procedure Store evidence complete
```

17.5 P4 - Storage V0 completion

Deliverables:

```
DiskPageStore
BufferPool integration
checkpoint records
cold snapshot publication
manifest switch
SegmentIndex startup
page corruption detection
```

Acceptance:

```
WAL-before-page-flush invariant tested
manifest switch crash tested
SegmentIndex avoids full cold scan
page torn write detected
```

17.6 P5 - Transaction and MVCC strengthening

Deliverables:

```
TransactionManager connected to heap/page runtime
ActiveSnapshotRegistry
MVCC row versions
rollback evidence
strict write locking for critical writes
V0 isolation policies
MVCC GC pins
```

Acceptance:

```
snapshot visibility deterministic
write conflict behavior defined
rollback restores table/index/map state
long reader blocks GC with quota evidence
```

17.7 P6 - QUIC application runtime

Deliverables:

```
runtime-quinn feature-gated server
application frames
mTLS development certificates
surface separation
bounded streams
backpressure
malformed frame tests
```

Acceptance:

```
Admin frame rejected on Application Surface
contract probing permission controlled
slow client triggers backpressure
no mutation through unsafe early-data path
```

17.8 P7 - IAM and audit durability

Deliverables:

```
UserPrincipal durable registry
CertificateIdentity durable registry
roles and permissions
surface scope policies
append-only audit ledger
break-glass policy
```

Acceptance:

```
disabled principal fails before transaction
permission denied emits durable audit
```

```
checksum chain detects deletion
break-glass bounded and auditable
```

17.9 P8 - Statistics and optimizer V0

Deliverables:

```
CPU statistics
StatsVersion candidate publication
bounded cost-based optimizer
PlanCacheKey strict identity
DecisionTrace
hysteresis
```

Acceptance:

```
stats candidate cannot publish without validation
plan uses catalog/stats/policy versions
DecisionTrace lists candidates considered and rejected
no learned component forces plan
```

17.10 P9 - Maps and analytics before GPU

Deliverables:

```
MapDescriptor
declared grain
summarizability checks
Immediate bounded Maps
Incremental delta Maps
Deferred and SnapshotOnly Maps
columnar segment layout
```

Acceptance:

```
Immediate Map cost bounded
Map maintenance cannot starve WAL
SnapshotOnly Map used for large analytics
aggregates respect declared grain
```

17.11 P10 - Optional GPU acceleration

Deliverables:

```
optional GPU crate
CPU fallback
GPU statistics prototype
GPU analytical Map scan prototype
GPU job traces
kill switches
```

Acceptance:

```
no C5 crate imports GPU runtime
gpu job cancellation safe
GPU-produced stats validated before publish
fallback reason recorded
```

17.12 P11 - HA/DR, backup, PITR, forensic drills

Deliverables:

```
WAL archive
cold snapshot backup
PITR exact LSN restore
cluster simulator
quorum/fencing tests
ForensicStart report
restore drills in CI lane
```

Acceptance:

```
replica is not treated as backup
promotion without quorum refused
restore to exact LSN tested
ForensicStart blocks application traffic
```

---

18. Risk register

Risk	Severity	Mitigation
Integration drift	Critical	Binding evidence everywhere, PR-sized batches, roadmap gates.
Unsafe unsoundness	Critical	Unsafe policy, Miri, fuzzing, review.
Panic in critical path	Critical	Typed errors, panic audit, crash tests.
Native struct serialization	Critical	Explicit codecs, review, fuzz.
Async cancellation corrupts state	High	Transaction guards, cancellation-safe boundaries.
Plan cache explosion	High	Bounded PlanClass and eviction.
Learned component overreach	High	Evidence-only and fallback.
GPU dependency in commit	Critical	Optional crate, import scans, policy tests.
QUIC custom complexity	High	Minimal V0 frames and typed errors.
Catalog central fragility	Critical	WAL-covered DefinitionBatch and recovery.

Risk	Severity	Mitigation
NVMe wear	High	Metrics, budgets, write amplification control.
Audit growth	Medium	Retention, compaction, cold tiers, signatures.
HA/DR split-brain	Critical	Quorum, fencing, epoch, no auto-promotion.
Supply-chain vulnerability	High	cargo-audit, cargo-deny, cargo-vet.
DDD object-persistence drift	High	Procedures and relational constraints remain canonical.

---

19. Required ADRs

Create or update these ADRs.

```
ADR-0001 Rust baseline and MSRV
ADR-0002 Workspace and crate boundaries
ADR-0003 Unsafe Rust policy
ADR-0004 Canonical binary format and endian policy
ADR-0005 WAL durability and group commit policy
ADR-0006 Page size and segment size defaults
ADR-0007 QUIC/RPC boundary and no gRPC
ADR-0008 Observability and trace schema
ADR-0009 GPU exclusion from commit path
ADR-0010 Supply-chain policy
ADR-0011 ContractHash canonicalization
ADR-0012 DefinitionBatch transactional publication
ADR-0013 Procedure Store runtime evidence
ADR-0014 StatsVersion publication policy
ADR-0015 PlanCacheKey and PlanClass policy
ADR-0016 Backup/PITR and forensic startup policy
```

---

20. Specification pack

Each specification must include purpose, scope, non-goals, data structures, invariants, serialization, state transitions, errors, security, observability, recovery behavior, compatibility, tests, and rejection criteria.

Prioritize:

```
ProcedureContract v0
TypeSystem v0
SRPL Grammar v0
SRPL Binder v0
Semantic IR v0
WalRecord v0
FileWal Segment v0
PageHeader/PageTrailer v0
```

```
DatabaseManifest v0
SegmentIndex v0
FrameHeader/RPC v0
TransactionStateMachine v0
CatalogObjectModel v0
DefinitionBatch v0
RecoveryReport v0
CrashRecoveryTestPlan v0
SecurityAdmission v0
AuditLedger v0
StatsObject v0
PlanCacheKey v0
ScenarioEvidence v0
GpuExecutionPolicy v0
```

---

21. Final normative recommendations

21.1 Rust

1. Keep repository MSRV at 1.85 until an ADR changes it.
2. Use Rust 1.95.0 as the verified 2026 stable development baseline.
3. Keep Rust 2024 Edition and resolver 3.
4. Deny `unsafe_op_in_unsafe_fn` in workspace lints.
5. Use stable production toolchain; use nightly only for tooling gates.

21.2 Architecture

1. Preserve four engine clusters and cross-cutting planes.
2. Do not create a God Engine.
3. Keep current crate layout until the durable vertical path is real.
4. Split optimizer, statistics, contract, codec, analytics, security, and admin crates only when pressure justifies it.
5. Keep the composition root thin.

21.3 Storage and transactions

1. No visible commit without durable WAL.
2. No page flush without WAL coverage.
3. No cold update-in-place.
4. No trust in isolated files.
5. Recovery must be tested by crash matrix, not argued by design.

21.4 Protocol and security

1. QUIC is transport; custom RPC is semantics.
2. Protobuf or canonical binary frames are acceptable; gRPC is not the application surface.
3. Validate contract, surface, IAM, and budget before transaction creation.
4. Separate Application, Administration, and HA/DR surfaces.
5. Emit durable audit for security decisions.

21.5 Optimization and hardware

1. Statistics are first-class engine data.
2. Procedure Store observes and feeds, but does not decide alone.
3. ScenarioEvidence remains non-authoritative.
4. Learned components are experimental until proven with fallback.
5. CPU acceleration must use runtime dispatch and scalar fallback.
6. GPU must remain optional, batch-only, and outside C5 paths.

21.6 Delivery order

The correct order is:

```
1. Truth and recovery.
2. Catalog and contracts.
3. SRPL and transaction-scoped execution.
4. Secure RPC.
5. Statistics and optimizer.
6. Maps and analytics.
7. Benchmark evidence.
8. Optional GPU acceleration.
9. HA/DR and production operations.
```

Do not shortcut toward GPU, learned optimization, or broad analytics before the durable Procedure path is real.

Appendix A - Current repository cross-check matrix

Repository evidence	Interpretation	Recommendation
`Cargo.toml` uses Edition 2024, resolver 3, rust-version 1.85.	Current MSRV is 1.85, not 1.95.	Adopt two-level baseline; bump only by ADR.
Workspace has 11 crates.	Modular foundation exists.	Do not reorganize before durable path is complete.
README states Procedure-only and not generic SQL server.	Doctrine is explicit in repo.	Keep no-SQL application-surface tests.
README exposes `vertical-v0`, `recovery-inspect`, `protocol-smoke`.	Local V0 verification exists.	Make these release-gate smoke checks.
Current state says recoverable vertical prototype, not production runtime.	Scope is correctly constrained.	Do not overstate readiness in docs.
Roadmap prioritizes durable path.	The next milestone is integration, not new feature breadth.	Focus on ProductStock durable path and catalog store.
QUIC/protocol crates exist.	Transport foundation exists.	Complete feature-gated server after admission gates.

Repository evidence	Interpretation	Recommendation
Benchmark and ScenarioEvidence scaffolds exist.	Advisory path exists.	Keep non-authoritative until optimizer V0.

Appendix B - Four-engine to crate map

Engine cluster	Current crates	Future optional crates
Contract and Language	`andromeda-catalog`, `andromeda-srpl`, `andromeda-proto`, `andromeda-core`	`andromeda-contract`, `andromeda-codec`
Transactional Truth	`andromeda-tx`, `andromeda-storage`, `andromeda-core`, `andromeda-observe`	`andromeda-wal` if storage grows too large
Surface and Governance	`andromeda-proto`, `andromeda-quic`, `andromeda-exec`, `andromeda-observe`, `andromeda-core`	`andromeda-security`, `andromeda-admin`
Adaptive Intelligence and Hardware	`andromeda-bench`, `andromeda-catalog`, `andromeda-core`, `andromeda-storage`, `andromeda-observe`	`andromeda-optimizer`, `andromeda-statistics`, `andromeda-analytics`, `andromeda-gpu`

Appendix C - Command cookbook

Core gates

```
cargo fmt --all -- --check
cargo check --workspace --locked
cargo test --workspace --locked
cargo clippy --workspace --all-targets --locked -- -D warnings
```

Extended gates

```
cargo nextest run --workspace --all-features
cargo test --doc --workspace
cargo audit
cargo deny check
cargo vet
```

Miri and fuzz

```
cargo +nightly miri test -p andromeda-core
cargo +nightly miri test -p andromeda-storage
cargo +nightly miri test -p andromeda-tx
cargo +nightly fuzz run wal_record -- -max_total_time=300
cargo +nightly fuzz run rpc_frame -- -max_total_time=300
cargo +nightly fuzz run srpl_parser -- -max_total_time=300
```

Local V0 commands

```
cargo run -p andromeda-cli -- vertical-v0 --wal "$env:TEMP\andromeda-v0-vertical.wal"
cargo run -p andromeda-cli -- recovery-inspect "$env:TEMP\andromeda-v0-vertical.wal"
cargo run -p andromeda-cli -- protocol-smoke --detail
```

Appendix D - External references

- Rust Blog. “Announcing Rust 1.95.0.” https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/
- Rust Edition Guide. “Cargo: Rust-version aware resolver.” https://doc.rust-lang.org/stable/edition-guide/rust-2024/cargo-resolver.html
- Rust Edition Guide. “Unsafe extern blocks.” https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-extern.html
- Rust Edition Guide. “Unsafe attributes.” https://doc.rust-lang.org/stable/edition-guide/rust-2024/unsafe-attributes.html
- Rust Reference. “Behavior considered undefined.” https://doc.rust-lang.org/reference/behavior-considered-undefined.html
- Rust Reference. “Unsafety.” https://doc.rust-lang.org/reference/unsafety.html
- Cargo Book. “Workspaces.” https://doc.rust-lang.org/cargo/reference/workspaces.html
- Cargo Book. “Profiles.” https://doc.rust-lang.org/stable/cargo/reference/profiles.html
- Miri. https://github.com/rust-lang/miri/
- Rust Fuzz Book. “Fuzzing with cargo-fuzz.” https://rust-fuzz.github.io/book/cargo-fuzz.html
- cargo-nextest. https://nexte.st/
- RustSec Advisory Database. https://rustsec.org/
- cargo-deny. https://embarkstudios.github.io/cargo-deny/
- cargo-vet. https://mozilla.github.io/cargo-vet/
- Tokio. https://tokio.rs/
- rustls. https://rustls.dev/
- RFC 9000. “QUIC: A UDP-Based Multiplexed and Secure Transport.” https://www.rfc-editor.org/rfc/rfc9000.html
- wgpu. https://wgpu.rs/
- Rust GPU. https://github.com/Rust-GPU/rust-gpu
- cudarc. https://docs.rs/cudarc/latest/cudarc/

Appendix E - Internal source basis

- `00_ANDROMEDA_INDEX_ET_MODE_DE_LECTURE.md`
- `01_DOCTRINE_LEXIQUE_ARCHITECTURE_CATALOGUE_MODELIZATION.md`
- `02_TYPE_SYSTEM_SRPL_PROCEDURES_MAPS.md`
- `03_TRANSACTION_WAL_MVCC_STORAGE_RECOVERY.md`
- `04_QUIC_RPC_SECURITY_HADR_OPERATIONS.md`
- `05_OPTIMIZER_STATS_ANALYTICS_HARDWARE_ROADMAP_SOURCES.md`
- `Moteur de stockage sécurisé.txt`
- `andromeda_rust_engineering_doctrine_2026.md`
- `Andromeda Rust 2026 Architecture and Engineering Guidance.pdf`
- `andromeda_modern_cpu_rust.pdf`
- `andromeda_modern_gpu_rust.pdf`
- `Rapport consolidé sur les SGBDR, ACID, les transactions, l'algebre relationnelle, la normalisation, les statistiques et l'indexation.pdf`
- `Corpus de reference pour les SGBDR, ACID, l'algebre relationnelle, la normalisation, les statistiques et les structures de recherche.pdf`
- `Fondation de SRPL pour un langage procedural relationnel strict.pdf`
- GitHub repository `AriusII/Andromeda`, branch `main`